

SWEN90010 Final Exam Review

Generated: 2026-06-23

Use for: final-night review before the 2026-06-24 exam

This packet is grounded in:

- outputs/notes/
- outputs/exams/2022-exam-practice.md
- outputs/exams/2023-pastpaper-practice.md
- outputs/exams/2024-pastpaper-practice.md
- outputs/exams/2026-predicted-mock-exam-14q-60marks.md
- outputs/review/schedule.md
- outputs/errors/error_log.md
- outputs/weaknesses/profile.md
- outputs/schemas/schema_ledger.md

0. What The Exam Is Really Testing

The exam is mostly testing whether you can apply schemas to new scenarios, not whether you can recite notes.

Expected domains:

Domain	What they usually ask you to do
Safety, HAZOP, FTA	identify hazards, causes, controls; fill HAZOP rows; build or critique fault trees
Fault tolerance	choose redundancy/voting designs; distinguish detection from masking
Security	identify trust boundaries; map STRIDE to properties; generate concrete threats
Alloy	write facts/functions/predicates/assertions; explain counterexamples and temporal operators
Ada/SPARK	translate C-like code; explain Constraint_Error; write safe preconditions; reason about contracts
Hoare logic	compute weakest preconditions; write loop invariants; prove init/preserve/exit
Pointers/separation logic	reason about aliasing; use points-to and separating conjunction

High-mark answer style:

1. Name the schema.
2. Apply it to the scenario.
3. Use exact notation where required.
4. Explain the marking reason in one or two sentences.

1. Safety Engineering, HAZOP, and Fault Trees

1.1 Safety Core

Must know:

- Safety means no unacceptable harm in the whole system.
- Correctness means implementation satisfies specification.
- Reliability means works without failure for a time/environment.
- A correct and reliable system can still be unsafe if the specification or design is unsafe.
- A hazard is an unsafe system state that can lead to an accident.
- Risk = severity plus likelihood/frequency.
- A safety case has claim, argument, and evidence.

Memorisable answer:

Safety is a whole-system property. Passing tests or satisfying a specification is not enough if the specification, sensor assumptions, human workflow, or operating environment can still lead to unacceptable harm.

Hazard-to-control frame:

Hazard: concrete unsafe state
Cause: why that state can arise
Severity: what harm could happen
Likelihood factor: concrete design/environment factor that makes it plausible
Control: mechanism that prevents, detects, or mitigates it
Evidence: tests, analysis, design documents, inspections, training, fault analysis

Example:

Hazard: medication pump delivers above maximum safe rate
Cause: wrong prescription input or failed rate sensor
Severity: catastrophic patient harm
Likelihood factor: no independent dose/rate check
Control: reject prescriptions above limit and stop pump if measured rate is too high
Evidence: test showing prescription 100 is rejected when limit is 10

Personal trap:

- Do not write the control as the safety claim.
- Claim = "pump will not deliver above safe limit".
- Control = "reject unsafe prescription".

1.2 HAZOP

HAZOP starts from intended design behaviour, not from a random hazard.

Template:

Design item: intended behaviour
Guideword: NO / MORE / LESS / OTHER THAN / EARLY / LATE / AS WELL AS
Deviation: system-specific wrong behaviour
Cause: why deviation happens
Consequence: what harm or unsafe state follows
Safeguard: control or mitigation

Memorisable answer:

HAZOP applies guidewords to an intended design item to generate deviations from intent, then records causes, consequences, and safeguards.

Common guideword examples:

Guideword	Example deviation
NO	no signal/message/authorisation is sent
LATE	signal arrives after it is needed
EARLY	signal arrives before it should
OTHER THAN	wrong patient, wrong medicine, wrong dose, wrong drawer
MORE	too much flow, too high dose, too many requests
LESS	too little braking, too low flow, incomplete message

1.3 Fault Tree Analysis

FTA starts from one top event/hazard and works backwards.

Gate rule:

OR = any one cause is sufficient
AND = all listed causes must occur together

Exam sensor pattern:

Function works if either sensor detects hazard.
Therefore detection fails only if sensor1 fails AND sensor2 fails.

Exam actuator/controller pattern:

Hazard occurs if brake/pump/nozzle/actuator fails OR command/detection fails.
Single points of failure usually appear under OR.
Redundant components usually appear under AND for loss of function.

Fault-tree skeleton:

```

top hazard
+ OR
  actuator failed
  command not sent
+ OR
  controller failed
  hazard not detected
+ AND
  sensor 1 failed
  sensor 2 failed

```

Personal trap:

- If either detector can detect the hazard, then one detector failing is not enough for missed detection.
- Both redundant detectors must fail: use AND.

2. Fault-Tolerant Design

Must know:

- Fault: underlying defect/cause.
- Error: wrong internal state or output caused by a fault.
- Failure: externally visible deviation from specification.
- Fault tolerance is bounded: the system continues to meet specification for a chosen finite number/type of faults.

Redundancy types:

Type	Meaning
Hardware redundancy	multiple physical units/sensors/processors
Software redundancy	diverse independent implementations
Information redundancy	parity, checksum, error-correcting codes
Time redundancy	retry, rollback, re-execute

Static pair vs majority voting:

2 components: can detect disagreement, cannot know which is correct.
 3 components: can mask 1 faulty output by majority voting.
 $N = 2m + 1$ components can mask m faulty units.

Memorisable answer:

A pair can detect disagreement, but it cannot mask one arbitrary failed unit because when outputs disagree there is no majority. To mask one non-Byzantine failed unit, use triple modular redundancy and majority voting.

Critical corrected point:

Minimum detectors to mask 1 detector failure = 3.
 Maximum failures tolerated by 3 detectors = 1.

Safety design trap:

- Changing stop-on-either-sensor to stop-only-if-both-sensors reduces false alarms but increases dangerous missed detections.
- In safety-critical stopping, false negative is usually worse than nuisance stop.
- Better: fail-safe stop on disagreement, or 3 sensors with voting/diagnostics.

3. Security: Trust Boundaries, STRIDE, and Attack Trees

3.1 Trust Boundaries

Trust boundaries are based on control/trust, not technology type.

Example:

Patient phone/app: not hospital-controlled
 Hospital API/database/identity/workstation: hospital-controlled
 Third-party provider: externally controlled

Relevant boundaries:

phone/app <-> hospital API
 third-party provider <-> hospital API

Memorisable answer:

A trust boundary appears where a message crosses from one control/trust region to another. Threat modelling focuses on those crossings.

3.2 STRIDE Mapping

Memorise exactly:

STRIDE	Threatens
Spoofing	Authentication
Tampering	Integrity
Repudiation	Non-repudiation / accountability
Information disclosure	Confidentiality
Denial of service	Availability
Elevation of privilege	Authorisation / access control

3.3 STRIDE Threat Sentence Frame

Use this every time:

An attacker/component [action] [target component/message/data], breaking [property].

Examples:

Category	High-mark threat
Spoofing	Attacker uses stolen session cookie to call API as a patient
Tampering	Compromised phone modifies medication-order request before API receives it
Repudiation	User denies order because audit logs are missing or unsigned
Disclosure	API/log/payment integration leaks prescription or identity data
DoS	Attacker floods API so legitimate medication orders cannot be processed
EoP	Patient exploits missing access check to call nurse-only endpoint

Personal trap:

- "Identity is leaked" alone is information disclosure.
- It becomes spoofing only if the attacker uses credentials/session/token to impersonate someone.
- For STRIDE generation, write the required number of threats. Do not stop after one example.

3.4 Attack Trees

Attack tree root = attacker goal.

Children = immediate alternative ways to achieve that goal.

Parent-child test:

The attacker can [root goal] by [child path].

Example root:

Access another user's private page

Good children:

- steal password
- steal session cookie
- change URL ID and exploit missing access check
- abuse backend/CDN bypass

Bad child:

- "view private page" because it repeats the root.

4. Alloy and Formal Specification

4.1 Alloy Basics You Must Write Correctly

Quantifiers:

```
all x : X | P[x]      // every x
some x : X | P[x]     // at least one x
one x : X | P[x]      // exactly one x
no x : X | P[x]       // no x
lone E                // zero or one element in expression E
```

Exactly-one predicate pattern:

```
one e : Employee | is_ceo[e]
```

Do not write:

```
all e : Employee | one is_ceo[e]
```

Relation direction:

```
supervisor : Employee -> lone Employee
State.supervisor[e] = u      // u is e's supervisor
State.supervisor.u           // employees supervised by u
```

Function constraint pattern:

```
always all f : FileName | lone State.files[f]
```

Meaning:

For fixed filename *f*, at most one File is mapped from *f*.
Not: at most one filename maps to a file.

4.2 State, Prime, and Temporal Operators

Core meanings:

```
var field      // field may change over time
field'         // next-state value of field
after P        // P holds in next state
always P       // P holds in every state of the trace
eventually P   // P holds now or in some future state
P ; Q         // P now, Q next
```

Personal trap:

- `after inv` means invariant holds in the next state after one operation.
- It does not mean "many states".
- It does not mean the object never changes.
- `always` is the many-states operator.

Trace-step trap:

1..1 steps = one state only, no meaningful pre/post transition.
One operation needs 2 states: pre-state and post-state.

For a transition assertion:

```
always all args |  
  operation[args] implies after postcondition
```

4.3 Alloy Operation Predicate Template

Every operation predicate has three jobs:

1. Preconditions: what must already be true?
2. Post-state value: what is created/updated?
3. Frame condition: what else stays the same?

Map update:

```
s.files' = s.files ++ (dest -> newFile)
```

Meaning:

next files mapping = old files mapping except dest now maps to newFile

Use `++` for override when one key should be replaced.

Use `+` after subtracting old pairs when updating a set-valued relation.

Set precondition pattern:

```
all sup : sups | State.staff_roles[sup] = Academic
```

Do not rely on:

```
Academic in State.staff_roles[sups]
```

because it does not clearly say every member of `sups` is Academic.

4.4 Common Alloy Answer Templates

Permissions lookup:

```
fun permissions[u : User, f : FileName, s : State] : set Permission {  
  s.files[f].acl[u]  
}
```

Copy/update relation:

```
pred copy[u, src, dest, s] {  
  some s.files[src]  
  Read in permissions[u, src, s]  
  some s.files[dest] => Write in permissions[u, dest, s]  
  
  one newFile : File | {  
    newFile.contents = s.files[src].contents  
    some s.files[dest] => newFile.acl = s.files[dest].acl  
    no s.files[dest] => newFile.acl = new_acl[u]  
    s.files' = s.files ++ (dest -> newFile)  
  }  
}
```

Write-safety interpretation:

If user lacks Write to f in the pre-state, f must not be changed by the transition.

But if creation is allowed, restrict to files that already exist:

```
some s.files[f] and Write not in permissions[u, f, s] implies
  s.files'[f] = s.files[f]
```

Read safety for copy:

```
copy[u, src, dest, s] implies Read in permissions[u, src, s]
```

Personal trap:

- Read safety is about whether the source is permitted to be read.
- It is not an unchangedness property, because reading may leave the source unchanged.

4.5 Alloy Counterexample Interpretation

If Alloy finds a strange counterexample:

```
Do not dismiss it.
Ask which missing fact, precondition, frame condition, or transition constraint allowed it.
```

If Alloy says no counterexample found:

```
It means none found within the chosen finite scope/step bound.
It is bounded confidence, not universal proof.
```

5. Ada, SPARK, and Ghost Code

5.1 Ada Core

Must know:

- Ada uses strong static typing.
- Different named types are incompatible even if their ranges look similar.
- Subtypes restrict an existing type.
- Runtime checks can raise `Constraint\_Error`.
- `.ads` is package specification; `.adb` is package body.
- `in`, `out`, `in out` make data flow explicit.
- Access types are pointer-like; `R'Access` passes a reference to `R`.
- Ada tasks start when created; procedures run when called.

Constraint_Error patterns:

```
Integer'Last + 1 -> overflow
Integer'First - 1 -> underflow
array index outside range -> bounds error
subtype assignment outside range -> range check failure
```

Direct C-to-Ada translation trap:

If question asks what Ada catches, first show the direct translation that exposes the type problem, then optionally give corrected code.

Example:

```
function Convert(F : Float) return Float is
  A : Integer := 2 + F;
begin
  return A;
```

```
end Convert;
```

This does not compile: Float expression assigned to Integer, then Integer returned as Float.

5.2 SPARK Safe Subset

Memorisable answer:

SPARK is a safe subset of Ada plus formal annotations. It restricts features such as dynamic allocation, unrestricted access types, aliasing, tasks, exceptions, goto, side-effectful functions, and recursion because these make memory use, control flow, data flow, or proof obligations harder for humans and tools to reason about.

Examples:

Restricted feature	Why it is hard to verify
dynamic allocation	memory use/lifetime unpredictable
tasks/concurrency	many interleavings
aliasing/access types	one update visible through another name
side-effectful functions	hidden state changes
exceptions/goto	non-local control flow
recursion	stack use depends on depth/input

Personal trap:

- Do not write only "SPARK avoids uncertainty".
- Add at least two concrete restricted features and why they matter.

5.3 Contracts and Modular Proof

Contract vocabulary:

```
Pre      // caller responsibility
Post     // implementation promise
X'Old    // value of X before call
F'Result // function result
Depends  // allowed data flow
```

SPARK prover call reasoning:

1. At call site, caller must prove callee Pre.
2. Inside callee, body must prove callee Post.
3. After call, caller may use only callee Post, not callee body.
4. Prover also checks possible runtime errors.

Modular proof trap:

If callee postcondition says only:

```
Post => I >= 0
```

then caller cannot prove:

```
Post => I > 0
```

unless it has a stronger contract or additional facts.

5.4 Weakest Error-Free Arithmetic Preconditions

Core rule:

The mathematical result must fit inside `Integer'First .. Integer'Last`.

Do not evaluate bounded ``A + B`` or ``A - B`` directly in a precondition if it may overflow.

Addition with `Big_Integer`:

```
Pre =>
  To_Big_Integer(Integer'First) <= To_Big_Integer(A) + To_Big_Integer(B)
  and then
  To_Big_Integer(A) + To_Big_Integer(B) <= To_Big_Integer(Integer'Last)
```

Subtraction with `Big_Integer`:

```
Pre =>
  To_Big_Integer(Integer'First) <= To_Big_Integer(A) - To_Big_Integer(B)
  and then
  To_Big_Integer(A) - To_Big_Integer(B) <= To_Big_Integer(Integer'Last)
```

Addition without `Big_Integer`:

```
Pre =>
  (if B > 0 then
    A <= Integer'Last - B
  elsif B < 0 then
    A >= Integer'First - B
  else
    True)
```

Personal trap:

- Need both upper and lower bounds.
- Need mathematical/unbounded arithmetic, not machine arithmetic inside the contract.

5.5 Ghost Code

Memorisable answer:

Ghost code is specification-only code used to help the prover reason about the real program. It is not needed for the deployed runtime result.

`Big_Integer`:

Used to talk about unbounded mathematical arithmetic before converting back to bounded `Integer`.

Ghost partial sums:

`Sum_Acc(A)(J)` = mathematical sum of array prefix up to index `J`.

Personal trap:

- ``A(I)`` is the current array element.
- ``Result`` is the runtime accumulator.
- ``Sum_Acc(A)(I-1)`` is the ghost mathematical prefix sum so far.
- Loop invariant links ``Result`` to the ghost model, not ``A(I)``.

6. Hoare Logic and Loop Invariants

6.1 Hoare Logic Core

Triple:

$\{P\} S \{Q\}$

Meaning:

If P holds before S , then Q holds after S , assuming S terminates.

This is partial correctness unless termination is proved separately.

Assignment axiom:

To prove Q after $x := E$, substitute E for future x in Q .

Example:

Goal after: $x = 5$
Statement: $x := x + 2$
Weakest precondition: $x + 2 = 5$, so $x = 3$

Consequence rule:

Can start with stronger precondition than needed.
Can conclude weaker postcondition than proved.

Conditional proof:

Prove then-branch under P and B .
Prove else-branch under P and not B .
Both must establish Q .

6.2 Weakest Precondition Scope

Personal trap from 2022 Q9:

For:

```
r := 0;
while (r+1)^2 <= X do
  r := r + 1
done
```

The weakest precondition before the whole program is:

$X \geq 0$

Reason:

r is overwritten by $r := 0$, so old r is irrelevant.
Negative X exits immediately with $r = 0$ and violates $0 \leq X$.

6.3 Loop Invariant Template

Loop proof obligations:

1. Initialization: I holds before first loop test.
2. Preservation: I and guard before body $\rightarrow I$ after body.
3. Exit: I and not guard $\rightarrow$ postcondition.

Invariant construction:

look at loop variables
look at postcondition
describe what has been processed so far
add bounds needed for safe indexing and exit reasoning

Do not put the whole postcondition into the invariant.

The part learned from `not guard` belongs to exit reasoning.

Square-root loop invariant:

$I = r * r \leq X$

The exit condition gives:

$\text{not } ((r+1)^2 \leq X) \Rightarrow X < (r+1)^2$

Array positive-count loop:

$I = 0 \leq i \leq N$ and $\text{count} = \text{positives}(i)$

Linear search loop:

$I =$
 $0 \leq i \leq N$
and $(\text{pos} = -1 \Rightarrow \text{notfound}(i))$
and $(\text{pos} \neq -1 \Rightarrow A[\text{pos}] = X \text{ and } 0 \leq \text{pos} \leq i - 1)$

Personal trap:

- During the loop, use checked prefix `i`, not whole array `N`.
- `notfound(N)` is available only at exit when `i = N`.

6.4 Loop Proof Sentence Frame

Use this structure in the exam:

Initialization:

After the initial assignments, no elements/iterations have been processed, so the prefix property holds for 0 and the I

Preservation:

Assume invariant and guard. Split into the relevant branch cases. Show the updated variables make the invariant true f

Exit:

At loop exit, not guard holds. Combine not guard with the invariant to derive the final postcondition.

For search preservation, split:

Case $A[i] = X$: pos becomes i; found branch holds.

Case $A[i] \neq X$: pos remains -1; notfound prefix extends from i to i+1.

7. Pointers and Separation Logic

7.1 Aliasing

Memorisable answer:

Two variables alias when they refer to the same memory location. If `X` and `Y` alias, writing through `X.all` also changes what is seen through `Y.all`.

Example:

```
*p = 5;  
*q = 6;
```

If p and q are separate: value at p is 5.

If p and q alias: value at p is 6.

Without a non-aliasing precondition: impossible to say.

Exam word:

alias

Do not call it "pointer for X" if the expected term is alias.

7.2 Pointer Assignment vs Dereference Assignment

Say the line aloud:

```
T := P;           // copy address/pointer value  
T.all := Q.all;   // copy integer through the addresses  
P := T;           // copy address/pointer value
```

Personal trap:

- ``P := Q`` copies the pointer.
- ``P.all := Q.all`` copies the value stored in the pointed-to cell.

7.3 Separation Logic

Points-to:

`p |-> x`

means pointer/address ``p`` points to a valid heap cell containing ``x``.

Separating conjunction:

`P * Q`

means P and Q hold on disjoint heap regions.

Important:

`p |-> 5 and q |-> 6`

does not necessarily give separation.

`p |-> 5 * q |-> 6`

does give separation/non-aliasing.

Common triple:

```
{ p |-> x * q |-> y }  
*p = *q;  
{ p |-> y * q |-> y }
```

Reason:

The command reads y from q and writes it into p. Because ``*`` says the cells are separate, q remains y.

Weakest precondition for writing fixed values:

$\text{exists } u, v. p \mid \rightarrow u * q \mid \rightarrow v$

Old values do not matter if the program overwrites both cells; separation and validity matter.

8. Personal High-Risk Checklist

Review these immediately before sleeping and once before entering the exam.

Safety and Fault Tolerance

- Masking one arbitrary sensor failure needs 3 sensors, not 2.
- Static pair detects disagreement; majority voting masks.
- In fault trees, redundant detection failure usually needs AND over all redundant sensors.
- Reducing false positives must not make the dangerous top hazard easier to reach.

STRIDE

- Spoofing must involve impersonation.
- Tampering must involve modifying data/message/record.
- Repudiation must involve denial because logs/evidence are missing or unreliable.
- Information disclosure must involve unauthorised seeing/leaking.
- DoS must block legitimate service.
- EoP must gain unauthorised permissions/access.
- Always use: actor + action + target + broken property.

Alloy

- ``sig`` declares fields; ``fact`` constrains existing fields.
- Follow relation arrows before writing ``lone``, ``some``, or lookup.
- For sets, use ``all x : set | property[x]``.
- For operation assertions, use ``after``.
- One operation step needs at least two states.
- Use whole-relation updates: ``relation' = old - removed + added`` or ``old ++ update``.
- Prime the field: ``State.supervisor``, not ``State'.supervisor``.
- Read safety is about source permission, not unchangedness.

Ada/SPARK

- Direct Ada translation may intentionally fail to compile and expose the C bug.
- A precondition is caller responsibility.
- A postcondition is implementation/callee promise.
- Modular proof uses callee contract, not callee body.
- Arithmetic preconditions need mathematical result in range.
- Do not put bounded ``A + B`` directly in a precondition if it may overflow.

Hoare Logic

- Whole-program weakest precondition accounts for assignments before the loop.
- Invariant says what is true during the loop, not the final condition learned from `not guard`.
- Processed prefix uses `i`, not `N`.
- Proof must cover initialization, preservation, and exit.
- Preservation for search/counting usually needs branch cases.

Pointers

- Different pointer names do not imply different heap cells.
- Without `p != q` or `\*`, aliasing may change the outcome.
- `p |-> v` is a heap assertion, not an assignment.
- `\*` in separation logic means non-overlapping heap regions.

9. Final-Night Drill Plan

Use this order because it matches the exam and protects high-risk marks.

90-minute version

1. 10 min: STRIDE table and two concrete threats per category.
2. 10 min: fault-tree AND/OR patterns plus 3-sensor masking rule.
3. 20 min: Alloy templates: function constraint, permissions, transition predicate, assertion with `after`.
4. 15 min: SPARK arithmetic preconditions using `Big_Integer` and modular proof obligations.
5. 20 min: loop invariant templates: square-root, count positives, linear search.
6. 10 min: pointer/separation-logic triples.
7. 5 min: read the personal high-risk checklist aloud.

30-minute emergency version

1. STRIDE mapping and sentence frame.
2. Alloy temporal/post-state template:

```
always all args | op[args] implies after postcondition
```
3. Arithmetic precondition:

```
Integer'First <= mathematical result <= Integer'Last
```
4. Loop proof:

```
init, preserve, exit
```
5. Separation logic:

```
p |-> x * q |-> y
```

10. Mini Self-Test

Cover the answer and say it from memory.

- STRIDE S/T/R/I/D/E properties:

Authentication, Integrity, Non-repudiation, Confidentiality, Availability, Authorisation.

- Minimum modules to mask m failures:

$N = 2m + 1$.

- Meaning of P' :

P' holds in the next state.

- Meaning of $\Box P$:

$\Box P$ holds in every state of the trace.

- Functional mapping for filename to file:

$\Box \text{always all } f : \text{FileName} \mid \text{one State.files}[f]$.

- Permission lookup:

$\Box \text{s.files}[f].\text{acl}[u]$.

- Operation postcondition template:

$\text{'field' = oldField ++ (key -> newValue)}$.

- Safe arithmetic precondition:

mathematical result within machine type bounds.

- Loop proof obligations:

initialization, preservation, exit.

- Search invariant prefix:

use $\text{notfound}(i)$, not $\text{notfound}(N)$.

- Pointer aliasing result after $*p=5; *q=6$:

impossible to say unless non-aliasing is known.

- Separation means:

$\Box$ says heap regions are disjoint.

11. If You Panic In The Exam

Use these fallback templates.

FTA:

Top hazard occurs if [single-point failure]
OR [command/detection lost].

Detection is lost only if
all redundant detectors fail.

HAZOP:

Guideword causes deviation from intended behaviour.
Deviation can lead to consequence.
Cause is component/process failure.
Safeguard detects, prevents, or fails safe.

STRIDE:

An attacker [verb] [component/message/data], breaking [property].

Alloy:

- preconditions
- post-state relation equation
- frame conditions

SPARK:

- Caller proves Pre.
- Callee proves Post.
- Prover checks runtime errors.
- Arithmetic must be in range using mathematical integers.

Loop invariant:

- Bounds + processed prefix meaning.
- Then prove `init`, `preserve`, `exit`.

Pointers:

- If aliasing is not ruled out, consider both `alias` and `non-alias` cases.
- Use ``*`` for separation.